Fiche sujet — Passerelle de notifications omnicanale avec bascule automatique

> Une API unique pour envoyer des SMS / e-mails (et à terme du push), avec routage configurable entre plusieurs fournisseurs et bascule automatique en cas de panne — sans réécrire le backend à chaque changement de provider.

1. Le problème

En entreprise, plusieurs applications (mobile et web) envoient des notifications via différents canaux : push, SMS, e-mail. Concrètement :

- Chaque application intègre **directement** les SDK / APIs des fournisseurs (Twilio, Resend, FCM…).
- À chaque changement de fournisseur, ou de canal, **l'infra doit être reconstruite** alors qu'elle existait déjà pour un autre provider.
- La logique de routage et de retry est **dispersée dans le code** de chaque application.
- Quand un fournisseur tombe en panne, c'est chaque application qui doit gérer l'échec — souvent mal, ou pas du tout.

En une phrase : chaque décision de fournisseur ou de canal se paie en réécriture du backend.

2. Le but de la solution

Séparer **quoi envoyer** (logique métier des applications) de **comment / chez qui l'envoyer** (routage vers les providers).

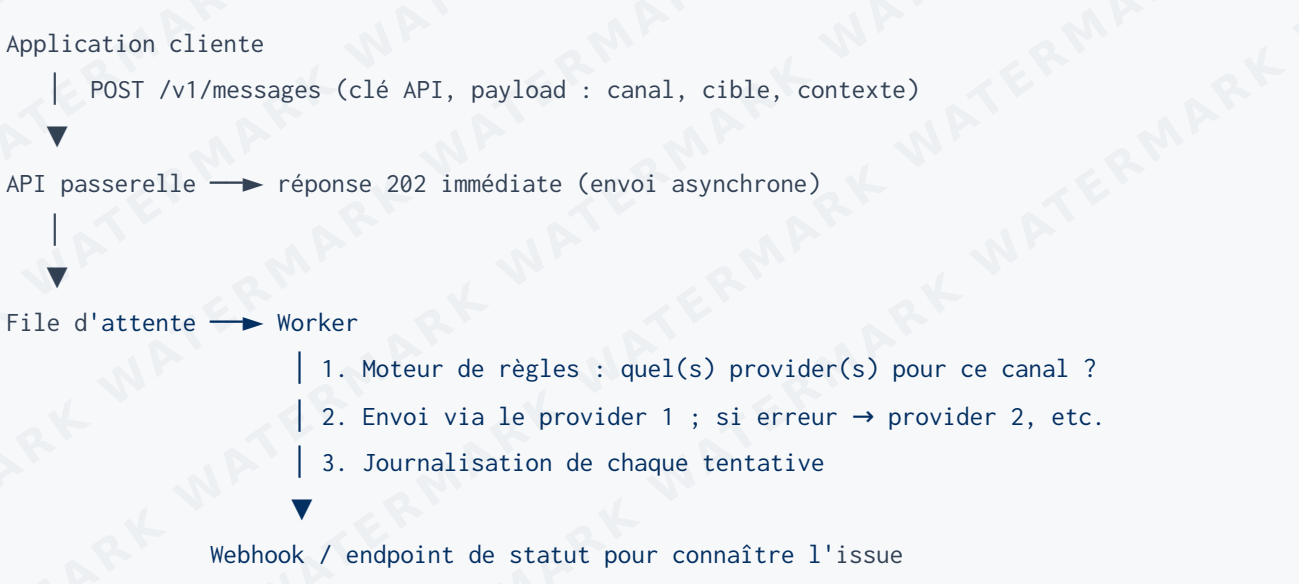
- Les applications ne parlent plus qu'**une seule API**.
- Changer de provider ou ajouter un secours devient une **configuration**
- Une panne de fournisseur est **absorbée automatiquement**, sans intervention.

3. La solution et comment elle résout le problème

Une **passerelle** (gateway) reçoit la demande d'envoi, décide quel provider doit l'envoyer, l'exécute en tâche de fond et journalise chaque tentative.

Problème	Comment la passerelle le résout
Couplage au fournisseur	Les apps appellent une seule API ; les providers se configurent côté passerelle
Réécriture à chaque changement	Routage = configuration (règles + ordres de fallback), pas du code applicatif
Panne d'un provider	Chaîne de bascule : provider principal → provider de secours, automatiquement
Erreurs invisibles	Journal des tentatives : chaque envoi, chaque provider testé, succès ou échec — et ces journaux sont **exportables vers la base de données de l'utilisateur** (webhooks / API), pour qu'il garde l'historique chez lui

4. Mise en œuvre (vue d'ensemble)



Points clés :

- **Envoyer** : une seule API REST, réponse immédiate, traitement en arrière-plan.
- **Configurer** : deux façons —
  - **endpoints admin / fichiers de configuration** : le mode naturel pour un développeur (config-as-code, versionnable) ;
  - **interface web du produit** : pour gérer les **credentials par utilisateur, par canal et par provider** sans passer par du curl — à plusieurs utilisateurs ou providers

- **Credentials fournisseurs** : saisis une fois par utilisateur / canal / provider, **chiffrés au repos**, révocables à tout moment chez le provider.
- **Règles** : priorité, pays, ou tout contexte fourni dans la requête → ordre des providers à essayer.
- **Journalisation** : chaque tentative (succès ou échec) est enregistrée, et récupérable par l'utilisateur via **webhooks ou API** pour être stockée dans **sa propre base de données**.

5. Périmètre — ce qui est fait / ce qui n'est pas fait

Fait (MVP)

- API d'envoi unique + clés d'authentification
- 2 canaux : **SMS + e-mail**, avec de preference **2 providers par canal**
- Bascule automatique + retries bornés
- Journal des tentatives (succès / échec)
- Statut par message + webhook signé
- Credentials chiffrés
- Configuration via endpoints admin / fichiers
- **Interface web minimale : gestion des credentials par utilisateur, canal et provider**
- Export des journaux de tentatives vers la base de l'utilisateur (webhooks / API)